

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 7

Datenformate

In der modernen Informatik, insbesondere im Bereich der Webentwicklung und verteilten Systeme, ist die persistente Speicherung und der Austausch von Datenstrukturen eine fundamentale Herausforderung. Applikationen halten Daten zur Laufzeit typischerweise in komplexen, sprachspezifischen Strukturen im Arbeitsspeicher, beispielsweise als Arrays oder Objekte in Java. Um diese Daten über ein Netzwerk zu übertragen (z. B. von einem Server zu einem Client) oder dauerhaft auf einem Datenträger zu speichern, wird ein Datenformat benötigt. Dieses kann binär oder textbasiert sein. Während binäre Formate zwar für die maschinelle Verarbeitung aufgrund ihrer Effizienz sehr gut geeignet sind, können sie nicht von Menschen einfach so gelesen werden. Textbasierte Formate haben dieses Problem nicht und haben sich daher vor allem im Web etabliert. Sie mit jedem Texteditor geöffnet, ihr Inhalt betrachtet und auch bearbeitet werden.

Dieses Kapitel beschäftigt sich mit den drei wichtigsten, textbasierten Datenformaten im Web: XML, JSON und YAML und stellt ihre Syntax und Einsatzgebiete dar.

7.1 XML

XML, kurz für *Extensible Markup Language*, ist eine vom W3C standardisierte Markup-Sprache [6]. Die Syntax ähnelt der bereits aus HTML bekannten – `<tag>Inhalt</tag>` – es gibt jedoch einige fundamentale Unterschiede.

7.1.1 Einführung

Bei XML handelt es sich um eine so genannte *Meta-Markup-Sprache*. Dies bedeutet, dass die Sprache dazu verwendet wird, eine Dokumentenbeschreibungssprache zu definieren. Da also mit XML eine eigene Sprache definiert wird, gibt es keine fixe Anzahl an Tags (wie bei HTML), sondern es können mit der Sprache beliebige Elemente festgelegt werden. Genau so, wie es der jeweilige Einsatzzweck erfordert. Dies wird auch durch den Ausdruck „extensible“ (deutsch: erweiterbar) im Namen der Sprache unterstrichen.

Eine solche selbst definierte Menge an eigenen Tags und Attributen wird auch eine *XML-Anwendung* (engl. „XML application“) genannt.

XML ist (wie auch HTML) eine Untermenge der *Standard Generalized Markup Lan-*

*guage*¹ (SGML). Da sich SGML jedoch als viel zu groß und komplex erwies, wurde beschlossen, mit XML eine einfachere, jedoch nicht minder flexible Meta-Markupsprache zu entwickeln.

Die Grammatik von XML ist sehr streng. Alle XML-Anwendungen müssen dieser Grammatik folgen (siehe Abschnitt 7.1.3).

Da XML, wie bereits einführend angemerkt, ein *Textformat* ist, ist es leicht lesbar und verarbeitbar. Dies eröffnet nicht nur die Möglichkeit, es plattformübergreifend verwenden zu können, auch eine gewisse Langlebigkeit ist dadurch garantiert. Binäre Datenformate haben oft das Problem, dass sie zwar auf einer Plattform verarbeitet werden, auf anderen Systemen jedoch nicht gelesen werden können. Und selbst wenn eine Datei prinzipiell gelesen werden kann, ist noch nicht garantiert, dass ihr Inhalt interpretiert werden kann. So gibt es von den Mondlandungen in den späten 1960er und frühen 1970er Jahren zwar noch Bänder mit den aufgezeichneten (binären) Daten, die Information, wie diese Datenströme zu lesen sind, existieren jedoch nicht mehr. Somit sind die Informationen effektiv verloren [3]. Textdaten sind nicht mit diesen Problemen konfrontiert. Zwar können hier unterschiedliche Kodierungen zum Einsatz kommen, doch diese lassen sich im schlimmsten Fall per Trial-and-Error herausfinden.

Der (meist) einfache und hierarchische Aufbau ermöglicht darüber hinaus eine gute Strukturierung von Dokumenten. Dies macht das Format universell einsetzbar.

7.1.2 Ein XML-Beispieldokument

Im Folgenden wird der grundsätzliche Aufbau von XML-Dokumenten beschrieben. Dies soll anhand eines Beispieldokuments geschehen, das eine mögliche XML-Anwendung darstellt. Nachfolgende Abschnitte greifen auf dieses Beispiel zu, um Aspekte von XML – wie etwa die Grammatik – zu erläutern.

Ein Rezept könnte in XML wie folgt dargestellt werden:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <rezept quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
3   <gericht>Der klassische Tiroler Knödel</gericht>
4   <zutaten>
5     <zutat>
6       <ingredienz>Semmelwürfel</ingredienz>
7       <menge>350</menge>
8       <einheit>g</einheit>
9     </zutat>
10    <zutat>
11      <ingredienz>Milch</ingredienz>
12      <menge>200</menge>
13      <einheit>ml</einheit>
14    </zutat>
15    <zutat>
16      <ingredienz>Eier</ingredienz>
17      <menge>4</menge>
18      <einheit>Stück</einheit>
19    </zutat>
20    <zutat>
21      <ingredienz>Speck</ingredienz>
22      <menge>100</menge>

```

¹<https://de.wikipedia.org/wiki/SGML>

```

23     <einheit>g</einheit>
24   </zutat>
25   <zutat>
26     <ingredienz>Bergsteigerwurst</ingredienz>
27     <menge>100</menge>
28     <einheit>g</einheit>
29   </zutat>
30   <zutat>
31     <ingredienz>Butter</ingredienz>
32     <menge>20</menge>
33     <einheit>g</einheit>
34   </zutat>
35   <zutat>
36     <ingredienz>Zwiebel</ingredienz>
37     <menge>1</menge>
38     <einheit>Stück</einheit>
39   </zutat>
40   <zutat>
41     <ingredienz>Petersilie</ingredienz>
42     <menge>10</menge>
43     <einheit>g</einheit>
44   </zutat>
45   <zutat>
46     <ingredienz>Salz</ingredienz>
47     <menge>1</menge>
48     <einheit>Prise</einheit>
49   </zutat>
50   <zutat>
51     <ingredienz>Muskatnuss</ingredienz>
52     <menge>1</menge>
53     <einheit>Prise</einheit>
54   </zutat>
55   <zutat>
56     <ingredienz>Mehl</ingredienz>
57     <menge>80</menge>
58     <einheit>g</einheit>
59   </zutat>
60   <zutat>
61     <ingredienz>Schnittlauch</ingredienz>
62     <menge>10</menge>
63     <einheit>g</einheit>
64   </zutat>
65 </zutaten>
66 <zubereitung>
67   <schritt>Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen
        lassen.</schritt>
68   <schritt>Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in
        zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten
        und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen
        Zutaten eine feste, aber nicht zu feuchte Masse kneten.</schritt>
69   <schritt>Mit befeuchteten Händen daraus Knödel formen.</schritt>
70   <schritt>Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat
        der Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die
        Knödelmasse kneten.</schritt>
71   <schritt>Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.
        </schritt>
72   <schritt>Tiroler Knödel mit Schnittlauch bestreut servieren.</schritt>

```

```
73   </zubereitung>
74 </rezept>
```

Das Dokument – ein Rezept für überaus wohlschmeckende Tiroler Knödel – besitzt ein Wurzelement namens `<rezept>`, das alle anderen Elemente beinhaltet. Dieses Element hat zusätzlich ein Attribut `quelle`, das nähere Informationen über die Herkunft des Rezepts spezifiziert. Darunter folgen die Elemente `<gericht>`, `<zutaten>` und `<zubereitung>`, die alle auf der gleichen Ebene liegen.

`<zutaten>` besteht aus einer Reihe von `<zutat>`-Elementen, welche alle jeweils ein `<ingredienz>`-, ein `<menge>`- und ein `<einheit>`-Element enthalten und damit die Zutat genauer spezifizieren.

Das Element `<zubereitung>` enthält einzelne `<schritt>`-Elemente, welche die einzelnen Zubereitungsschritte textuell beinhalten.

7.1.3 XML-Grammatik

Wie in Abschnitt 7.1.2 ersichtlich, besteht ein XML-Dokument prinzipiell aus Text, der mit Tags wie etwa `<ingredienz>` ausgezeichnet wurde. Jeder Tag kann auch noch Attribute enthalten. Dies unterscheidet sich in der grundsätzlichen Verwendung nicht von HTML, der erste offensichtliche Unterschied ist jedoch der Tag-Name `<ingredienz>`, der in HTML so nicht existiert, da HTML nur eine endliche Menge an Tags definiert. Während also XML wesentlich freizügiger bei der Wahl der Tag-Namen ist, ist es wesentlich strikter, wenn es darum geht, wie diese geschrieben und platziert werden. Alle XML-Dokumente müssen nämlich *well-formed* sein, d. h. strikten grammatikalischen Regeln entsprechen.

Ein Wurzelement

Jedes XML-Dokument hat genau ein *Wurzelement* (engl. „root element“), also ein Element, das kein Elternelement besitzt. Im Beispiel in Abschnitt 7.1.2 ist `<rezept>` das Wurzelement.

Paarweise Tags

Jedes Element muss in XML ein öffnendes und schließendes Tag haben, also etwa `<ingredienz>Mehl</ingredienz>`. Standalone-Elemente wie in HTML (z. B. `<br>`) gibt es nicht. Hat ein Element keinen Inhalt, so wird es in sich geschlossen. In XML-Schreibweise (d. h. XHTML, siehe Abschnitt 7.1.6) lautet das Element für Zeilenumbrüche `<br />`. Die Schreibweise `<br></br>` ist äquivalent dazu, aber eher ungebräuchlich.

Keine überkreuzenden Tags

Wie in HTML gilt auch bei XML, dass Tags in der umgekehrten Reihenfolge geschlossen werden müssen, in der sie geöffnet wurden. Überkreuzungen sind also nicht erlaubt und führen zu einem Fehler. Somit gilt:

- **Korrekt:** `<a><b>...</b></a>`.
- **Inkorrekt:** `<a><b>...</a></b>`.

Attribute: Anführungszeichen und Einzigartigkeit

Jedes Element kann zwischen 0 und n *Attribute* besitzen. Ein Attribut ist wie in HTML eine Zusatzangabe in der Form `name="wert"`. Im Gegensatz zu HTML müssen in XML alle Attributwerte in Anführungszeichen stehen (auch dann, wenn der Wert des Attributs keine Leerzeichen enthält). Möglich sind doppelte Anführungszeichen, also z. B. `quelle="https://..."`, oder auch einfache Anführungszeichen (z. B. `quelle='https://...'`).

Darüber hinaus darf jedes Element nur ein Attribut mit demselben Namen haben. Würde das Rezept aus Abschnitt 7.1.2 auf mehreren Seiten im Web existieren und sollen mehrere solcher Quellen über das Attribut `quelle` angegeben werden, so wäre dies nicht möglich. Es darf nur ein Attribut `quelle` beim Element `<rezept>` geben. Die Lösung in diesem Fall wäre, die Quelle nicht als Attribut, sondern als Kindelement von Rezept anzugeben, denn davon darf es durchaus mehrere geben (wie es z. B. auch bei `<zutat>` ersichtlich ist). Die Diskussion, ob Daten als Attribut oder Element abgebildet werden sollen, ist eine sehr oft geführte. Als Faustregel kann folgendes gelten:

Attribute werden für Metainformationen, die ein Element ergänzen, verwendet. Elemente enthalten immer die eigentliche Information.

Generell sollte aber je nach XML-Anwendung selbst entschieden werden.

Namen von Elementen und Attributen

Bei der Verwendung von *Namen in XML* ist immer die Groß- und Kleinschreibung zu beachten. Die Tags `<rezept>` und `<Rezept>` sind zwei unterschiedliche Elemente.

Darüber hinaus können Namen alle alphanumerischen Zeichen enthalten, auch nicht-englische wie etwa deutsche Umlaute, kyrillische Zeichen oder griechische Symbole. Weiters sind der Underscore (`_`), der Bindestrich (`-`) und der Punkt (`.`) erlaubt. Wichtig ist, dass der Name eines Elements mit einem Buchstaben oder Underscore beginnen muss. Andere Interpunktionszeichen oder Leerzeichen sind nicht erlaubt.

Die folgenden Elemente sind also alle well-formed (aus [3]):

- `<Drivers_License_Number>98NY32</Drivers_License_Number>`,
- `<month-day-year>7/23/2001</month-day-year>`,
- `<first_name>Alan</first_name>`,
- `<_4-lane>I-610</_4-lane>`,
- `<téléphone>011 33 91 55 27 55 27</téléphone>`.

Inkorrekt sind hingegen die folgenden Elemente:

- `<Driver's_License_Number>98NY32</Driver's_License_Number>` (Apostroph),
- `<month/day/year>7/23/2001</month/day/year>` (Schrägstrich),
- `<first name>Alan</first name>` (Leerzeichen),
- `<4-lane>I-610</4-lane>` (beginnt mit Zahl).

Entitäten, PCDATA und CDATA

Der Inhalt eines Elements kann beliebig gestaltet werden, es gibt jedoch fünf Zeichen, die durch so genannte *Entitäten* (bekannt bereits aus HTML) ersetzt werden müssen:

- `<`: `<`; (wird sonst für den Beginn eines Tags angenommen),
- `>`: `>`; (wird sonst für das Ende eines Tags angenommen),
- `&`: `&`; (wird sonst als Beginnzeichen einer Entität interpretiert),
- `"`: `"`; (wird beim Attributwert als doppeltes Anführungszeichen erkannt),
- `'`: `'`; (wird beim Attributwert als einfaches Anführungszeichen erkannt).

Wie aus der Aufzählung ersichtlich ist, müssen diese fünf Elemente durch Entitäten ersetzt (engl. „escaped“) werden, da sie beim maschinellen Verarbeiten der XML-Datei (Parseen, mehr dazu im zweiten Semester in der Web Technologies Vorlesung) fälschlicherweise für etwas gehalten werden (z. B. Beginn eines Tags), das sie gar nicht darstellen. Es kann etwa ein mathematischer Ausdruck einfach nur `5 > 3` enthalten – wird das „Größer“-Zeichen nicht durch eine Entität ersetzt, so „stolpert“ der XML-Parser. Dieser sieht nämlich standardmäßig das gesamte Dokument als PCDATA (Parsed Character Data) an, also Inhalt, der vom Parser abgearbeitet wird. Dabei werden Verschachtelungen und andere XML-Eigenschaften erkannt und richtig interpretiert.

Enthält aber ein XML-Element nun viele Zeichen, die escaped werden müssen, so ist dies mitunter sehr aufwändig. Abhilfe schaffen kann ein so genannter CDATA-Abschnitt. CDATA steht für Character Data, auf Deutsch etwa „Zeicheninhalt“. Dieser wird vom Parser einfach als Inhalt wahrgenommen, er versucht darin nicht, etwas zu interpretieren. In einem CDATA-Block können also Zeichen wie `<`, `>` oder `&` gefahrlos verwendet werden. Ein CDATA-Block beginnt immer mit einer spitzen Klammer, danach folgt ein Ausrufezeichen, gefolgt von einer eckigen Klammer, dem Wort „CDATA“ und einer erneuten eckigen Klammer. Geschlossen wird der CDATA-Abschnitt mit zwei eckigen und einer spitzen Klammer.

Im folgenden XML-Ausschnitt wird etwa in einem XML-Dokument ein HTML-Element erläutert. Überall dort, wo zu ersetzende Zeichen vorkommen, wird ein CDATA-Block verwendet und somit das Escapen hinfällig:

```
1 <element>
2   <name><![CDATA[<h1>]]></name>
3   <description>Heading Level 1</description>
4   <example><![CDATA[<h1>Hello World</h1>]]></example>
5 </element>
```

Kommentare

Kommentare in XML werden in derselben Syntax wie in HTML verfasst. Das öffnende Kommentar wird mit spitzer Klammer, Ausrufezeichen und zwei Bindestrichen, das schließende Kommentar mit zwei Bindestrichen und einer spitzen Klammer erstellt. Kommentare dienen für Anmerkungen im Quellcode und sollten von Parsern oder anderen Programmen nicht ausgewertet werden. Sie dürfen überall im Inhaltsbereich des Dokuments vorkommen, nur nicht innerhalb von Tags. Das folgende Beispiel zeigt einen XML-Kommentar:

```
1 <!-- Das Root-Element -->
```

Processing Instructions

Während Kommentare als Zusatzinformationen für Autor*innen oder Leser*innen einer XML-Datei dienen, ermöglichen es die so genannten *Processing Instructions* Anweisungen an die Software zu geben, welche die XML-Datei maschinell verarbeitet (z. B. der Parser). Processing Instructions beginnen mit spitzer Klammer und einem Fragezeichen und werden durch ein Fragezeichen und eine spitze Klammer abgeschlossen.

Um etwa für XML-Dokumente anzugeben, dass sie von Suchmaschinen (Robots) indiziert werden sollen, aber keinen enthaltenen Links folgen sollen, ist die folgende Processing Instruction möglich:

```
1 <?robots index="yes" follow="no"?>
```

Die Einbindung von PHP-Code (sowohl in XML als auch in HTML) erfolgt ebenfalls über eine Processing Instruction – mehr dazu dann im zweiten Semester.

```
1 <?php
2 phpinfo();
3 ?>
```

XML-Deklaration

Von der Syntax her identisch zu den Processing Instructions ist die *XML-Deklaration* (auch XML-Präfix). Wenn sie verwendet wird, muss sie immer am Anfang eines XML-Dokuments stehen und gibt dem Parser Auskunft, welche XML-Version und Zeichenkodierung verwendet werden. Ist sie nicht vorhanden, muss der Parser dies selbst herausfinden, was nicht immer akkurat gelingt. Das XML-Beispiel in Abschnitt 7.1.2 beinhaltet ebenfalls eine Deklaration in Zeile 1. Dies ist die universellste Art der Deklaration: Version 1.0 [6] ist der meistgenutzte² Standard, die Zeichenkodierung UTF-8 erlaubt die höchste Flexibilität. Sie sieht daher wie folgt aus:

```
1 <?xml version="1.0" encoding="UTF-8"?>
```

7.1.4 Well-formed und Valid

Ein XML-Dokument gilt als *well-formed* (deutsch: „wohlgeformt“ (sic!)), wenn es die soeben erwähnten Grammatikregeln erfüllt. Am wichtigsten sind die folgenden, noch einmal kurz zusammengefassten. Darüber hinaus gibt es natürlich noch eine große Anzahl weiterer Faktoren, die eingehalten werden müssen bzw. die falsch gemacht werden können, um ein Dokument nicht mehr well-formed erscheinen zu lassen:

1. Jedes Start-Element muss ein entsprechendes End-Element besitzen.
2. Elemente können verschachtelt, dürfen aber nie überkreuzt werden.
3. Es darf nur genau ein Wurzelement geben.
4. Attributwerte müssen unter Anführungszeichen gesetzt werden.
5. Ein Element darf nicht zwei Attribute mit demselben Namen haben.

²Obwohl Version 1.1 [7] des Standards existiert, ist dieser nur in den seltensten Fällen nötig und die Verwendung von Version 1.0 empfohlen. Unter <https://www.ibiblio.org/xml/books/effectivexml/chapters/03.html>, einem Ausschnitt aus [2], wird erläutert warum.

6. Kommentare und Processing Instructions dürfen nicht innerhalb von Tags auftreten.
7. Die Zeichen `<` und `&` dürfen nicht unescaped im Inhalt von Elementen vorkommen.

Ein Dokument gilt in weiterer Folge als *valid* (deutsch: valide, gültig), wenn es einerseits well-formed ist und andererseits einem definierten Schema entspricht. Ein Schema ist entweder eine Document Type Declaration (kurz: DTD oder DOCTYPE) oder ein XML Schema. Beides wird nun näher in Abschnitt 7.1.5 erläutert.

7.1.5 Validierung durch Schemas

Während XML durch die freie Wahl der Elementnamen für Entwickler*innen sehr flexibel ist, bedeutet diese Flexibilität für Applikationen, die mit XML umgehen müssen, mitunter eine Herausforderung. Die Anwendung muss beim maschinellen verarbeiten genau wissen, wie eine XML-Applikation aufgebaut ist, ansonsten kann sie mit den Daten nicht arbeiten. Auch zu überprüfen, ob die zugrunde liegende XML-Datei korrekt ist, kann essentiell sein, um eine funktionierende Verarbeitung zu ermöglichen.

Dieses Dilemma lösen *Schemas*, die beschreiben, wie ein XML-Dokument aufgebaut sein muss. Dabei gibt es zwei Möglichkeiten: Document Type Definitionen (DTDs) und XML Schemas.

Document Type Definition

Die *Document Type Definition*, kurz *DTD* oder auch DOCTYPE genannt, ist in einer eigenen, formalen Sprache verfasst und beschreibt den Aufbau einer XML-Applikation. Es werden Aussagen getroffen wie „das Wurzelement ist `<rezept>`“ oder „jedes `<zutat>`-Element beinhaltet die Elemente `<ingredienz>`, `<menge>` und `<einheit>`“. Ein validierender Parser kann dann den Aufbau der XML-Datei mit der DTD vergleichen und auf Korrektheit überprüfen.

Aufbau einer DTD: Eine Document Type Definition ist ein einfaches Textdokument mit der Erweiterung `.dtd`. Sie enthält verschiedene Arten von Deklarationen. Eine Deklaration beginnt immer mit einer spitzen Klammer und einem Ausrufezeichen, danach folgt der Typ der Deklaration mit den nötigen Zusatzangaben, danach endet die Deklaration mit einer spitzen Klammer:

```
1 <!TYP Zusatzangaben>
```

Folgende Typen können spezifiziert werden:

- **ELEMENT:** Deklariert ein Element, d. h. ein Tag.
- **ATTLIST:** Deklariert Attribute zu einem Element.
- **ENTITY:** Deklariert eine Konstante.
- **NOTATION:** Deklariert nicht-XML-Dateien, um diese in XML verwenden zu können. Dies wird jedoch selten verwendet und daher in diesem Abschnitt auch nicht näher erläutert.

Element-Deklaration: *Element-Deklarationen* werden verwendet, um Elemente einzuführen. Die generelle Syntax lautet wie folgt:

```
1 <!ELEMENT title (content)>
```

Der Typ ist also **ELEMENT**, **title** wird durch den Namen des zu deklarierenden Elements ersetzt und die Angabe von **content** in Klammern spezifiziert, welcher Inhalt in diesem Element enthalten ist.

Leere Elemente: Leere Elemente werden definiert, indem anstelle von **content** der Wert **EMPTY** angegeben wird. Soll also beispielsweise ein XML-Element `<foo />` deklariert werden, so muss in der DTD folgendes angegeben sein:

```
1 <!ELEMENT foo (EMPTY)>
```

Elemente mit Textinhalt: Elemente, die nur Text beinhalten sollen, können mit dem Wert **#PCDATA** (Parsed Character Data, siehe Abschnitt 7.1.3) für **content** deklariert werden. Um ein Element `<anmerkung>Das hier ist wichtig!</anmerkung>` (Textinhalt kann beliebig sein) zu ermöglichen, ist folgender Eintrag in der DTD notwendig:

```
1 <!ELEMENT anmerkung (#PCDATA)>
```

Elemente mit Kindelementen: Sollen Elemente andere Elemente enthalten, werden diese in der Klammer von **content** angegeben. Dabei gibt es verschiedene Möglichkeiten:

Die Angabe von **ANY** beim Element `<random>` erlaubt jedes beliebige Kindelement:

```
1 <!ELEMENT random (ANY)>
```

Die Angabe eines Elements erlaubt eben nur dieses. Darf ein Element `<familie>` nur das Element `<kind>` enthalten, so wird dies folgendermaßen deklariert:

```
1 <!ELEMENT familie (kind)>
```

Mehrere Kindelemente können durch Komma separiert angegeben werden. Soll das Element `<auto>` die Kindelemente `<hersteller>`, `<typ>` und `<baujahr>` haben, so wird dies wie folgt deklariert:

```
1 <!ELEMENT auto (hersteller, typ, baujahr)>
```

Gibt es eine Auswahl an möglichen Kindelementen, d. h. ein Element `<sport>` kann entweder das Kindelement `<fußball>`, oder das Kindelement `<tennis>`, oder das Kindelement `<laufen>` beinhalten, so kann dies durch den „Oder“-Operator `|` angegeben werden:

```
1 <!ELEMENT sport (fußball|tennis|laufen)>
```

Schließlich kann durch Zusatzangaben die Häufigkeit des Auftretens spezifiziert werden:

- **?**: Optionales Auftreten, das Kindelement kann 0 oder 1 Mal vorkommen.
- **+**: Das Kindelement kann 1 bis n mal vorkommen (d. h. mindestens einmal).
- *****: Das Kindelement kann 0 bis n mal vorkommen (d. h. beliebig oft oder auch gar nicht).

Über Klammerung ist zudem noch eine Gruppierung möglich. Soll ein Element `<nachricht>` deklariert werden, das ein Element `<absender>`, mindestens ein Element `<empfänger>`, ein Element `<inhalt>` und optional entweder das Element `<bcrypt>` oder

`<argon2i>` (zur Angabe eines möglichen Hash-Wertes) beinhaltet, würde dies wie folgt deklariert:

```
1 <!ELEMENT nachricht (absender, empfänger+, inhalt, (bcrypt|argon2i)?)>
```

Attribut-Deklaration: Mit einer *Attribut-Deklaration* wird angegeben, welche Attribute ein Element besitzt. Die generelle Syntax lautet wie folgt:

```
1 <!ATTLIST tag attr type default>
```

Der Typ lautet **ATTLIST**, **tag** bezeichnet das Element, für das die Attribute angegeben werden. Mit **attr** wird der Name des Attributs spezifiziert, **type** gibt die Art des Attributs an. **default** spezifiziert, ob ein Attribut verlangt wird oder ausgelassen werden kann bzw. ob es einen Standardwert besitzt.

Folgende Werte sind für den Typ des Attributs möglich:

- **CDATA:** Das Attribut kann jeden beliebigen Textinhalt aufweisen.
- **ID:** Das Attribut muss einen eindeutigen Inhalt aufweisen (wie es in HTML für das `id`-Attribut üblich ist).
- **IDREF:** Das Attribut muss auf ein anderes vom Typ `ID` verweisen (wie das `for`-Attribut bei `<label>`, das auf `id` von `<input>` verweist).
- **IDREFS:** Das Attribut enthält eine durch Leerzeichen getrennte Liste von Werten, die auf Attribute vom Typ `ID` verweisen.
- **ENTITY:** Das Attribut verweist auf eine Konstante.
- **ENTITIES:** Das Attribut enthält die Namen mehrerer Konstanten.
- **NOTATION:** Das Attribut enthält den Namen einer Notation.
- **NMTOKEN:** Das Attribut enthält einen Wert, der der XML-Namensgebung entspricht (keine Leerzeichen, nur alphanumerische Zeichen etc.– wie in Abschnitt 7.1.3 erwähnt).
- **NMTOKENS:** Das Attribut enthält mehrere Werte, die der XML-Namensgebung entsprechen.
- **Aufzählung:** Nur die in der Liste angegebenen Werte sind erlaubt.

Folgende Werte sind für **default** möglich:

- **#IMPLIED:** Das Attribut ist optional und es ist kein Standardwert vorgegeben.
- **#REQUIRED:** Das Attribut muss immer angegeben werden, es ist aber kein Standardwert vorgegeben.
- **#FIXED "value":** Das Attribut muss angegeben werden und muss den Wert `value` haben.
- **"value":** Das Attribut ist optional. Wird es nicht angegeben, wird der mit `value` angegebenen Standardwert angenommen.

Attribute mit Textinhalt: Soll das Element `<song>` ein Attribut `title` besitzen, welches jeden textuellen Wert annehmen kann, aber angegeben werden muss, lautet die Deklaration wie folgt:

```
1 <!ATTLIST song title CDATA #REQUIRED>
```

Attribute mit Aufzählungen: Um eine vordefinierte Liste an Werten anzugeben, werden diese in Klammer mit dem „Oder“-Operator (senkrechter Strich, Pipe) aneinander gereiht: (wert1|wert2|wert3|...)

Soll für ein Element <vehikel> ein Attribut **typ** existieren, das nur die Werte **pkw**, **lkw**, **flugzeug** oder **boot** annehmen soll und bei dem der Standardwert **pkw** lautet, falls es nicht angegeben wird, so lautet die Deklaration:

```
1 <!ATTLIST vehikel typ (pkw|lkw|flugzeug|boot) "pkw">
```

Entität-Deklaration: *Entitäten* sind eine Art Konstante, bei der ein vordefinierter Text hinterlegt werden kann. Dies ist dann nützlich, wenn derselbe Text oft im Dokument vorkommt. Durch das Definieren einer Entität wird er einmal zentral definiert und kann überall dann referenziert werden. Es gibt eine Reihe von Entitäten, die wichtigsten sollen hier erläutert werden:

Generelle Entitäten: Allgemeine Entitäten liegen immer in der Form **&name;** vor. Bekannt sollten etwa **&** für **&** oder **<** für **<** sein. Um eine eigene Entität zu definieren, kommt folgende Syntax zur Anwendung:

```
1 <!ENTITY name "text">
```

name bezeichnet den Namen der Entität, das initiale **&** und abschließende **;** dürfen nicht enthalten sein. **text** ist der Inhalt, der für die Entität eingesetzt werden soll.

Um eine Entität **&fhoee;** mit dem Wert **Copyright 2025 FH OÖ** zu erzeugen, ist folgende Deklaration notwendig:

```
1 <!ENTITY fhoee "Copyright 2025 FH OÖ">
```

Parameter-Entitäten: Parameter-Entitäten können verwendet werden, um eine vordefinierte Parameterliste für eine Element-Deklaration zu erstellen. Dies ist dann interessant, wenn genau dieselbe Liste oft verwendet wird. Die allgemeine Form lautet:

```
1 <!ENTITY % name "text">
```

Angenommen, verschiedene Elemente können entweder Text oder auch das Element <description>, beides aber optional enthalten, so muss die Definition dessen (**#PCDATA|description**)* lauten. Die Beschreibung dieses Inhalts kann nun in eine Parameter-Entität „verpackt“ werden:

```
1 <!ENTITY % varcontent "(#PCDATA|description)*">
```

Nun können verschiedene Elemente definiert werden, die genau diese Inhaltsbeschreibung zugewiesen bekommen:

```
1 <!ELEMENT letter %varcontent>
2 <!ELEMENT info %varcontent>
```

Beispiel-DTD: Im Folgenden soll nun die DTD für das Tiroler Knödel-Rezept aus Abschnitt 7.1.2 dargestellt werden.

```

1 <!ELEMENT rezept (gericht, zutaten, zubereitung)>
2 <!ATTLIST rezept quelle CDATA #REQUIRED>
3
4 <!ELEMENT gericht (#PCDATA)>
5
6 <!ELEMENT zutaten (zutat)+>
7
8 <!ELEMENT zubereitung (schritt)+>
9
10 <!ELEMENT zutat (ingredienz, menge, einheit)>
11
12 <!ELEMENT schritt (#PCDATA)>
13
14 <!ELEMENT ingredienz (#PCDATA)>
15 <!ELEMENT menge (#PCDATA)>
16 <!ELEMENT einheit (#PCDATA)>

```

Das Element `<rezept>` enthält genau jeweils ein Element der Typen `<gericht>`, `<zutaten>` und `<zubereitung>` und besitzt ein verpflichtendes Attribut `quelle`, dessen Inhalt Text ist.

Das Element `<gericht>` enthält beliebigen Text. `<zutaten>` enthält mindestens ein `<zutat>`-Element. Das Element `<zubereitung>` enthält mindestens ein `<schritt>`-Element. `<zutat>` enthält je ein Element der drei Typen `<ingredienz>`, `<menge>` und `<einheit>`. `<schritt>` enthält beliebigen Text. Die Elemente `<ingredienz>`, `<menge>` und `<einheit>` enthalten jeweils beliebigen Text.

DTDs einbinden: Die Angabe einer DTD erfolgt in der XML-Datei direkt nach der XML-Deklaration, also in der zweiten Zeile. Die Syntax ist analog zu der aus HTML und lautet wie folgt:

```
1 <!DOCTYPE name SYSTEM url>
```

`!DOCTYPE` zeigt die Angabe einer DTD an, `name` ist der eindeutige Name. Dieser muss immer mit dem Namen des Wurzelements übereinstimmen, im Falle des Tiroler Knödel-Rezepts ist dies `rezept`. `SYSTEM` gibt an, dass die DTD an einem externen Ort liegt, dass es sich jedoch nicht um eine öffentliche DTD handelt. Mit `url` wird die Adresse zur DTD im Web angegeben, es kann jedoch auch eine relative URL sein. Die Angabe der DTD für das Tiroler Knödel-Rezept kann wie folgt aussehen:

```
1 <!DOCTYPE rezept SYSTEM "https://www.hochleitner.cc/dtds/rezept.dtd">
```

Öffentliche DTDs: Standardisierte DTDs verwenden nicht den Identifikator `SYSTEM`, sondern `PUBLIC`. Dies bedeutet, dass es sich um bekannte DTDs handelt, die beim Validieren etwa nicht online „nachgeschlagen“ werden müssen, da sie dem Validator in der Regel bekannt sind. Ein Beispiel hierfür ist der DOCTYPE für XHTML 1.0:

```

1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

```

XML Schema Definition (XSD)

Lange Zeit waren die DTDs der Standard zur Validierung von XML-Daten, sie wurden jedoch bald aufgrund fehlender Datentypen und eigener Syntax weitgehend durch die

XML Schema Definition (XSD) [8] ersetzt.

Aufbau eines Schemas: Ein XML Schema ist wie selbst ein XML-Dokument, das Mechanismen zur Typisierung (String, Integer, Date), Kardinalität (wie oft darf ein Element vorkommen) und Inhaltsmodellierung bietet. Es wird in einer Datei mit der Erweiterung *.xsd* gespeichert. Das Schema beginnt wie ein normales XML-Dokument mit einer XML-Deklaration, danach folgt das Wurzelement:

```
1 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
2   ...
3 </xs:schema>
```

Tags und auch Datentypen in XSD beginnen immer mit dem Präfix **xs:** (für XML Schema). Die Attributangabe spezifiziert die Schema-Definition, in diesem Fall XML Schema Version 1.0.

Innerhalb dieses Wurzelements folgen weitere Angaben:

- **<xs:element>**: Deklariert ein Element, d. h. ein Tag.
- **<xs:attribute>**: Deklariert Attribute zu einem Element.
- **<xs:complexType>**: Deklariert ein Element mit Unterelementen/Attributen.
- **<xs:simpleType>**: Beschreibt einen einfachen Datentyp ohne Kindelemente, kann jedoch Einschränkungen enthalten.
- **<xs:sequence>**, **<xs:choice>**, **<xs:all>**: Beschreiben Kindelemente und deren Reihenfolge bzw. Alternativen.

Element-Deklaration: Mit einer *Element-Deklaration* wird festgelegt, welche Elemente im XML-Dokument vorkommen dürfen und welcher Typ diesen Elementen zugeordnet ist. Die allgemeine Syntax lautet:

```
1 <xs:element name="gericht" type="xs:string"/>
```

Dies deklariert ein Element **<title>**, das beliebigen Textinhalt (Datentyp String) besitzt.

Einfache Elemente: Elemente, die nur Inhalt enthalten sollen, erhalten einen einfachen Datentyp wie **xs:string**, **xs:integer**, **xs:decimal**, **xs:boolean** oder ähnliche Typen des XML-Schemas:

```
1 <xs:element name="menge" type="xs:decimal"/>
```

Komplexe Elemente: Enthält ein Element weitere Kindelemente oder Attribute, so wird es als *komplexer Typ* definiert. Dies geschieht mit **xs:complexType** und einem Inhaltsmodell für die Reihenfolge mit **xs:sequence**:

```
1 <xs:element name="zutat">
2   <xs:complexType>
3     <xs:sequence>
4       <xs:element name="ingredienz" type="xs:string"/>
5       <xs:element name="menge" type="xs:decimal"/>
6       <xs:element name="einheit" type="xs:string"/>
7     </xs:sequence>
```

```

8   </xs:complexType>
9 </xs:element>

```

Wie bei DTDs steuern zusätzliche Angaben die Kardinalität der Elemente:

- `minOccurs="0"`: Optionales Auftreten.
- `maxOccurs="unbounded"`: Beliebig oft wiederholbar.
- `minOccurs="1"`: Mindestens einmal.

Attribut-Deklaration: Attribute werden mittels `xs:attribute` deklariert. Hier kommen ebenfalls einfache Datentypen zum Einsatz:

```

1 <xs:attribute name="id" type="xs:ID" use="required" />

```

Das Attribut `use` steuert die Verfügbarkeit:

- `use="optional"`: Das Attribut kann angegeben werden.
- `use="required"`: Das Attribut muss angegeben werden.
- `use="prohibited"`: Das Attribut ist verboten.

Einfache Typen: Mit `xs:simpleType` können eigene Datentypen definiert werden, etwa um Aufzählungen oder Einschränkungen festzulegen.

Aufzählungen: Aufzählungen legen fix definierte Werte für Inhalte fest:

```

1 <xs:simpleType name="EinheitTyp">
2   <xs:restriction base="xs:string">
3     <xs:enumeration value="g"/>
4     <xs:enumeration value="ml"/>
5     <xs:enumeration value="Stück"/>
6   </xs:restriction>
7 </xs:simpleType>

```

Dies erzeugt einen Datentyp namens „EinheitTyp“, der auf Text (String) basiert und die Werte „g“, „ml“ und „Stück“ haben darf.

Einschränkungen: Einschränkungen definieren Grenzen für Werte:

```

1 <xs:simpleType name="MengeTyp">
2   <xs:restriction base="xs:decimal">
3     <xs:minExclusive value="0"/>
4   </xs:restriction>
5 </xs:simpleType>

```

Dies definiert einen Datentyp namens „MengeTyp“, der auf einer Zahl (Decimal) basiert, aber nur positive Dezimalzahlen größer als 0 erlaubt.

Beispiel-XSD: Im Folgenden nun das XML Schema für das Rezept aus Abschnitt 7.1.2 dargestellt:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3   <xs:element name="rezept">
4     <xs:complexType>
5       <xs:sequence>

```

```

6      <xs:element ref="gericht"/>
7      <xs:element ref="zutaten"/>
8      <xs:element ref="zubereitung"/>
9      </xs:sequence>
10     <xs:attribute name="quelle" type="xs:anyURI" use="required"/>
11   </xs:complexType>
12 </xs:element>
13
14 <xs:element name="gericht" type="xs:string"/>
15
16 <xs:element name="zutaten">
17   <xs:complexType>
18     <xs:sequence>
19       <xs:element maxOccurs="unbounded" ref="zutat"/>
20     </xs:sequence>
21   </xs:complexType>
22 </xs:element>
23
24 <xs:element name="zubereitung">
25   <xs:complexType>
26     <xs:sequence>
27       <xs:element name="schritt" type="xs:string" maxOccurs="unbounded"/>
28     </xs:sequence>
29   </xs:complexType>
30 </xs:element>
31
32 <xs:element name="zutat">
33   <xs:complexType>
34     <xs:sequence>
35       <xs:element name="ingredienz" type="xs:string"/>
36       <xs:element name="menge" type="xs:decimal"/>
37       <xs:element name="einheit" type="xs:string"/>
38     </xs:sequence>
39   </xs:complexType>
40 </xs:element>
41 </xs:schema>

```

Dieses Schema legt fest, dass ein `<rezept>`-Element ein verpflichtendes Attribut `quelle` besitzt und die Elemente `<gericht>`, `<zutaten>` und `<zubereitung>` genau in dieser Reihenfolge enthalten muss. Die Zutatenliste enthält mindestens ein `<zutat>`-Element, welches wiederum die Elemente `<ingredienz>`, `<menge>` und `<einheit>` in genau dieser Reihenfolge enthält. `<zubereitung>` umfasst beliebig viele `<schritt>`-Elemente. Datentypen schränken die Inhalte der Elemente zusätzlich ein.

XML Schema einbinden: Damit ein XML-Dokument gegen ein XML-Schema validiert werden kann, muss im Dokument selbst ein Verweis auf die entsprechende `.xsd`-Datei angegeben werden. Im Gegensatz zu DTDs erfolgt diese Einbindung nicht über einen `<!DOCTYPE>`-Eintrag, sondern über ein spezielles Attribut im Wurzelement des XML-Dokuments.

Zur Einbindung wird der sogenannte *XML Schema Instance*-Namespace (`xsi`) eingebunden und anschließend mit dem Attribut `xsi:noNamespaceSchemaLocation` auf die Datei verwiesen, welche das Schema enthält. Die Syntax lautet wie folgt:


```

1 <rootElement xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="schema.xsd">
2 ...
3 </rootElement>

```

`xmlns:xsi` deklariert das XML Schema Instance-Namespace, welches die Attribute zur Einbindung von Schemas bereitstellt. Mit `xsi:noNamespaceSchemaLocation` wird die URL oder der relative Pfad zur Schema-Datei angegeben. Dieses Attribut wird verwendet, wenn das XML-Dokument *keine Namespaces* verwendet (was bei einfachen XML-Strukturen typischerweise der Fall ist).

Für das Tiroler Knödel-Rezept sieht die Einbindung wie folgt aus:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <rezept xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:noNamespaceSchemaLocation="https://www.hochleitner.cc/schemas/rezept.xsd"
4     "
5     quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
6 </rezept>

```

7.1.6 XML-Applikationen

XML-Applikationen sind verschiedene Markupsprachen, die mittels XML definiert wurden. Bekannte Vertreter sind u. a.:

- **SVG**: Scalable Vector Graphics³ – das bekannteste und am weitesten verbreitete Format zur Darstellung von Vektorgrafiken im Web.
- **MathML**: Mathematical Markup Language⁴ – Ein Dokumentenformat zur Darstellung mathematischer Formeln im Web.
- **XHTML**: Extensible Hypertext Markup Language⁵ – die Neuformulierung von HTML 4.01 in XML. In den 2000er Jahren der Standard, mit dem Aufkommen von HTML5 obsolet geworden.
- **SMIL**: Synchronized Multimedia Integration Language⁶ – eine Auszeichnungssprache für zeitsynchronisierte, multimediale Inhalte.
- **RDF**: Resource Description Framework⁷ – eine Sprache zur Beschreibung von Informationen über Objekte (Metadaten) im Web. Wichtig für das Semantic Web⁸.
- **RSS**: Really Simple Syndication⁹ – ein Format zur Veröffentlichung von Änderungen auf Websites (bekannt als RSS-Feeds).

7.2 JSON

XML ist zweifelsohne ein vielseitig verwendbares und auch gut menschenlesbares Datenformat. Daraus ergibt sich aber auch ein entscheidender Nachteil: XML ist sehr

³https://de.wikipedia.org/wiki/Scalable_Vector_Graphics

⁴https://de.wikipedia.org/wiki/Mathematical_Markup_Language

⁵https://de.wikipedia.org/wiki/Extensible_Hypertext_Markup_Language

⁶https://de.wikipedia.org/wiki/Synchronized_Multimedia_Integration_Language

⁷https://de.wikipedia.org/wiki/Resource_Description_Framework

⁸https://de.wikipedia.org/wiki/Semantic_Web

⁹[https://de.wikipedia.org/wiki/RSS_\(Web-Feed\)](https://de.wikipedia.org/wiki/RSS_(Web-Feed))

ausschweifend – im Englischen „verbose“ (wörtlich: langatmig, wortreich). Es werden also viele Zeichen (d.h. Bytes) für die Beschreibung der Daten (die Tags) verwendet, was vor allem beim maschinellen Lesen (unnötigen) Overhead bedeutet. Aus diesem Grund hat sich zu Beginn der 2000er Jahr ein weiteres Datenformat etabliert: JSON.

7.2.1 Allgemein

JSON steht für *JavaScript Object Notation*¹⁰ [1]. Dabei handelt es sich um ein kompaktes Datenformat, dessen Syntax ein Subset der Objekt-Literal-Notation von JavaScript ist (siehe dazu Vorlesung 9). D.h., in JSON notierte Daten ähneln JavaScript-Objekten (daher auch der Name), sind aber kein JavaScript-Code. JSON ist sprachunabhängig und wird in allen populären Programmier- und Scriptsprachen unterstützt. Wird JSON in Dateien abgespeichert, so handelt es sich um Textdateien mit der Endung `.json`. Sehr oft werden JSON-Daten aber einfach als Zeichenkette (String) zwischen zwei Endstellen übertragen, etwa zwischen Client und Server, wenn so genannte REST-Schnittstellen zum Einsatz kommen (mehr dazu im zweiten Semester in der Vorlesung zu Web Technologies).

7.2.2 Ein JSON-Beispieldokument

Als Beispiel soll wiederum das Knödelrezept aus Abschnitt 7.1.2 dienen. In JSON sieht dieses etwa wie folgt aus:

```
1 {
2   "gericht": "Der klassische Tiroler Knödel",
3   "quelle": "https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386",
4   "zutaten": [
5     {
6       "ingredienz": "Semmelwürfel",
7       "menge": 350,
8       "einheit": "g"
9     },
10    {
11      "ingredienz": "Milch",
12      "menge": 200,
13      "einheit": "ml"
14    },
15    {
16      "ingredienz": "Eier",
17      "menge": 4,
18      "einheit": "Stück"
19    },
20    {
21      "ingredienz": "Speck",
22      "menge": 100,
23      "einheit": "g"
24    },
25    {
26      "ingredienz": "Bergsteigerwurst",
27      "menge": 100,
28      "einheit": "g"
29    }
30  ]
31 }
```

¹⁰<https://www.json.org/>

```

29   },
30   {
31     "ingredienz": "Butter",
32     "menge": 20,
33     "einheit": "g"
34   },
35   {
36     "ingredienz": "Zwiebel",
37     "menge": 1,
38     "einheit": "Stück"
39   },
40   {
41     "ingredienz": "Petersilie",
42     "menge": 10,
43     "einheit": "g"
44   },
45   {
46     "ingredienz": "Salz",
47     "menge": 1,
48     "einheit": "Prise"
49   },
50   {
51     "ingredienz": "Muskatnuss",
52     "menge": 1,
53     "einheit": "Prise"
54   },
55   {
56     "ingredienz": "Mehl",
57     "menge": 80,
58     "einheit": "g"
59   }
60   {
61     "ingredienz": "Schnittlauch",
62     "menge": 10,
63     "einheit": "g"
64   }
65 ],
66 "zubereitung": [
67   "Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.",
68   "Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in
69   zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten
70   und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen
71   Zutaten eine feste, aber nicht zu feuchte Masse kneten.",
72   "Mit befeuchteten Händen daraus Knödel formen.",
73   "Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat der
74   Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die Knö
  
```

Der gesamte JSON-Datensatz ist ein Objekt – eine Sammlung von Name/Wert-Paaren, gekennzeichnet durch geschwungene Klammern ({ }). Die Namen, die immer Zeichenketten (Strings) sind, werden dabei auch oft als Schlüssel bezeichnet. Auf der

obersten Ebene gibt es vier Schlüssel: **gericht**, **quelle**, **zutaten** und **zubereitung**.

Der Wert zu jedem Schlüssel steht nach einem Doppelpunkt und kann verschiedene Datentypen annehmen:

- **gericht** und **quelle**: Die Werte sind hier jeweils eine Zeichenkette (String).
- **zutaten** und **zubereitung**: Der Wert ist jeweils eine geordnete Liste von Werten (ein Array), gekennzeichnet durch eckige Klammern (`[ ]`). Bei den Zutaten sind die einzelnen Einträge der Liste wiederum Objekte (mit eigenen Name/Wert-Paaren), bei der Zubereitung sind die Werte einfache Zeichenketten.

Hier zeigt sich der grundsätzliche Unterschied zu XML: Es kommen keine Tags zum Einsatz, vielmehr werden Datenstrukturen (die aus der Programmierung in vielen Sprachen bekannt sind) eingesetzt. Die Benennung erfolgt über den Namen in den Objekten. Attribute als Zusätze zu einzelnen Daten gibt es nicht, diese müssen als eigenes Name/Wert-Paar abgebildet werden.

7.2.3 JSON-Syntax

JSON basiert, wie im Beispiel ersichtlich, auf zwei Datenstrukturen:

- **Objekt**: Eine Sammlung von Name/Wert-Paaren.
- **Array**: Eine geordnete Liste von Werten.

Darüber hinaus gibt es noch einfache Wert-Typen – Literale genannt:

- **String**: Text, also eine Folgen von Unicode-Zeichen.
- **Number**: Eine Dezimalzahl.
- **true** und **false**: Boolesche Werte, die *wahr* oder *falsch* repräsentieren.
- **null**: Gibt an, dass explizit *kein* Wert vergeben wurde.

Objekte

Objekte sind ungeordnete Sets von Name-Wert-Paaren. Ein JSON-Objekt beginnt mit einer linken, geschwungenen Klammer (`{`) und endet mit einer rechten, geschwungenen Klammer (`}`). Der Name ist immer ein String, steht unter doppelten Anführungszeichen (`"`) und ist vom Wert durch einen Doppelpunkt (`:`) getrennt. Mehrere Name-Wert-Paare sind durch Beistriche (`,`) voneinander getrennt. Leerzeichen und Zeilenumbrüche zwischen den einzelnen Trennzeichen sind möglich und können der Lesbarkeit helfen.

So kann ein Objekt in einer Zeile geschrieben werden:

```
1 { name1: wert1, name2: wert2 }
```

Meist jedoch, werden die Klammern und Name/Wert-Paare in eigene Zeilen gesetzt:

```
1 {
2   name1: wert1,
3   name2: wert2
4 }
```

Namen/Schlüssel können nur Zeichenketten (Strings) sein, Werte können Objekte, Arrays oder Literale (String, Zahl, **true**, **false**, **null**) sein.

Im Tiroler Knödel-Rezept ist jede Zutat ein Objekt mit drei Schlüssel. Die Werte sind zwei mal ein String und einmal eine Zahl:

```
1 {  
2   "ingredienz": "Semmelwürfel",  
3   "menge": 350,  
4   "einheit": "g"  
5 }
```

Arrays

Arrays sind geordnete Listen von Werten. Sie beginnen mit einer linken, eckigen Klammer ([]) und enden mit einer rechten, eckigen Klammer (]). Die einzelnen Werte eines Arrays sind mit Beistrichen (,) voneinander getrennt. Wieder können Leerzeichen und Zeilenumbrüche beliebig verwendet werden.

Somit kann ein Array einzilig geschrieben werden:

```
1 [ wert1, wert2, wert3 ]
```

Meist wird hier jedoch eine Aufteilung auf einzelne Zeile aufgrund der Lesbarkeit bevorzugt:

```
1 [  
2   wert1,  
3   wert2,  
4   wert3  
5 ]
```

Werte eines Arrays können wiederum Objekte, Arrays oder Literale sein.

Im Tiroler Knödel Rezept ist die Zubereitung ein Array, dessen Werte Zeichenketten sind:

```
1 "zubereitung": [  
2   "Semmelwürfel mit Milch ...",  
3   "Kleinstwürfelig geschnittenen Speck ...",  
4   "Mit befeuchteten Händen ...",  
5   "Einen davon als Probeknödel ...",  
6   "Erst wenn die Masse kompakt genug ist, ...",  
7   "Tiroler Knödel mit Schnittlauch bestreut servieren."  
8 ]
```

Literale

Als *Literale* werden einfache Werte bezeichnet. Dazu gehören Zeichenketten (String), Zahlen (Number), boolesche Werte (**true/false**) und **null**.

Strings: Strings werden unter doppelten Anführungszeichen geschrieben. Zeilenumbrüche müssen über Escapezeichen wie `\n` realisiert werden.

```
1 {  
2   "gericht": "Der klassische Tiroler Knödel"  
3 }
```

Zahlen: Zahlen (Numbers) werden ohne Anführungszeichen geschrieben. Unterstützt werden nur Dezimalzahlen (keine Oktal-, Hexadezimal- oder Binärzahlen).

```
1 {  
2   "menge": 350  
3 }
```

true, **false** und **null**: **true** und **false** sind so genannte boolesche Werte oder auch Wahrheitswerte. Sie repräsentieren die zwei Zustände wahr und falsch und werden direkt, ohne Anführungszeichen geschrieben. **null** ist ein spezieller Wert, der einfach „nichts“ oder „leer“ repräsentiert. Diese drei Werte kommen im Tiroler Knödel-Rezept nicht vor, könnten aber wie folgt verwendet werden:

```
1 {  
2   "aus_österreich": true,  
3   "vegetarisch": false,  
4   "bewertung": null  
5 }
```

In diesem Fall wäre gekennzeichnet, dass das Rezept aus Österreich stammt, nicht vegetarisch ist und noch keine Bewertungen vorliegen.

7.2.4 Validierung durch JSON-Schema

Auch JSON-Dokumente können anhand eines Schemas validiert werden. Der etablierte Standard hierfür ist *JSON Schema*¹¹. Dabei handelt es sich um ein selbst in JSON verfasstes Format, mit dem definiert werden kann, welche Inhalte ein JSON-Dokument haben muss und welche Datentypen zum Einsatz kommen.

Aufbau eines Schemas

Ein JSON-Schema wird in einer Textdatei mit der Endung `.schema.json` gespeichert und enthält zunächst ein Objekt mit folgenden Einträgen (die hier gezeigten Werte orientieren sich immer am Tiroler Knödel-Rezept).

```
1 {  
2   "$schema": "https://json-schema.org/draft/2020-12/schema",  
3   "$id": "https://www.hochleitner.cc/schemas/rezept.schema.json",  
4   "title": "Rezept",  
5   "description": "Ein Kochrezept mit Zutaten und Zubereitungsschritten.",  
6   "type": "object",  
7   "properties": { ... },  
8   "additionalProperties": false,  
9   "required": [ ... ]  
10 }
```

Die Schlüssel bedeuten dabei folgendes:

- **\$schema**: Ist ein Schema-Schlüssel und spezifiziert die verwendete Schema-Version.
- **\$id**: Ebenfalls ein Schema-Schlüssel. Er gibt eine eindeutige Bezeichnung für das Schema an. Hierfür wird meist ein eindeutiger URL gewählt.
- **title**: Bezeichnet, was das Schema definiert.
- **description**: Eine inhaltliche Beschreibung des Schemas.

¹¹<https://json-schema.org/>

- **type**: Spezifiziert den Datentyp, in dem die gesamte JSON-Datei vorliegen muss – hier ein Objekt.
- **properties**: Definiert die Liste an Namen/Schlüsseln, die in diesem Objekt beinhaltet sein können.
- **additionalProperties**: Legt fest, ob neben den durch **properties** festgelegten Schlüsseln noch weitere erlaubt sind.
- **required**: Entscheidet, welche Schlüssel zwingend benötigt werden.

Eigenschaften deklarieren

Eigenschaften (engl. „properties“) sind die Namen oder Schlüssel einer JSON-Datei. Um sie zu definieren, kommt die **properties**-Angabe zum Einsatz:

```
1 "properties": {  
2   "gericht": {  
3     "description": "Das Gericht oder der Name des Rezepts.",  
4     "type": "string",  
5     "minLength": 1  
6   },  
7   "quelle": {  
8     "description": "Die Quelle oder URL des Rezepts.",  
9     "type": "string",  
10    "format": "uri"  
11  },  
12 }
```

Dies definiert die beiden Eigenschaften **gericht** und **quelle**. Beide sind ein String, ersterer hat die Einschränkung, dass er mindestens 1 Zeichen lang sein muss, zweiterer muss ein gültiger URI sein. Auf dieselbe Art und Weise können auch andere Typen wie Arrays oder Objekte festgelegt werden. Da letztere wiederum aus Eigenschaften bestehen, müssen diese dann wieder verschachtelt definiert werden:

```
1 "zutaten": {  
2   "description": "Die Liste der Zutaten für das Rezept.",  
3   "type": "array",  
4   "minItems": 1,  
5   "items": {  
6     "type": "object",  
7     "properties": {  
8       "ingredienz": {  
9         "description": "Eine einzelne Zutat des Rezepts.",  
10        "type": "string",  
11        "minLength": 1  
12      },  
13      "menge": {  
14        "description": "Die Menge der Zutat.",  
15        "type": "number",  
16        "exclusiveMinimum": 0  
17      },  
18      "einheit": {  
19        "description": "Die Einheit der Menge.",  
20        "type": "string",  
21        "enum": [  
22          "g",
```

```

23         "ml",
24         "Stück",
25         "Prise"
26     ]
27 }
28 }
29 }
30 }

```

zutaten ist ein Array, das mindestens einen Eintrag haben muss. Die Einträge selbst sind wieder Objekte, die aus den Eigenschaften **ingredienz**, **menge** und **einheit** bestehen. Einheit ist dabei als **enum** definiert, was nur die vier hier angegebenen Werte zulässt.

Zusätzliche und verpflichtende Werte

Schließlich kann mit den Angaben **additionalProperties** und **required** noch spezifiziert werden, ob neben den im Schema angegebenen Schlüsseln noch weitere erlaubt sind und welche von den angegeben vorkommen *müssen*, d. h. verpflichtend sind. Diese Angaben werden nach der **properties**-Definition gemacht und können so aussehen:

```

1 "properties": {
2   ...
3   "additionalProperties": false,
4   "required": [
5     "gericht",
6     "quelle",
7     "zutaten",
8     "zubereitung"
9   ]
10 }

```

7.2.5 Beispiel-Schema

Im Folgenden nun das JSON-Schema für das Rezept aus Abschnitt 7.2.2 dargestellt:

```

1 {
2   "$schema": "https://json-schema.org/draft/2020-12/schema",
3   "$id": "https://www./schemas/rezept.schema.json",
4   "title": "Rezept",
5   "description": "Ein Kochrezept mit Zutaten und Zubereitungsschritten.",
6   "type": "object",
7   "properties": {
8     "gericht": {
9       "description": "Das Gericht oder der Name des Rezepts.",
10      "type": "string",
11      "minLength": 1
12    },
13    "quelle": {
14      "description": "Die Quelle oder URL des Rezepts.",
15      "type": "string",
16      "format": "uri"
17    },
18    "zutaten": {
19      "description": "Die Liste der Zutaten für das Rezept.",

```



```
20     "type": "array",
21     "minItems": 1,
22     "items": {
23       "type": "object",
24       "properties": {
25         "ingredienz": {
26           "description": "Eine einzelne Zutat des Rezepts.",
27           "type": "string",
28           "minLength": 1
29         },
30         "menge": {
31           "description": "Die Menge der Zutat.",
32           "type": "number",
33           "exclusiveMinimum": 0
34         },
35         "einheit": {
36           "description": "Die Einheit der Menge.",
37           "type": "string",
38           "enum": [
39             "g",
40             "ml",
41             "Stück",
42             "Prise"
43           ]
44         }
45       },
46       "additionalProperties": false,
47       "required": [
48         "ingredienz",
49         "menge",
50         "einheit"
51       ]
52     },
53   },
54   "zubereitung": {
55     "description": "Die Zubereitungsschritte des Rezepts.",
56     "type": "array",
57     "minItems": 1,
58     "items": {
59       "type": "string",
60       "minLength": 1
61     }
62   },
63 },
64 "additionalProperties": false,
65 "required": [
66   "gericht",
67   "quelle",
68   "zutaten",
69   "zubereitung"
70 ]
71 }
```

7.2.6 JSON-Schema verwenden

Im Gegensatz zu XML-Dokumenten, gibt es in JSON keine Möglichkeit, das Schema in der JSON-Datei anzugeben. Ein Tool, wie etwa eine IDE, muss also wissen, dass es ein Schema gibt, wo sich dieses befindet und es laden und damit arbeiten.

In PhpStorm kann dies unter **File** » **Settings** » **Schemas and DTDs** » **JSON Schema Mappings** geschehen. Mit **+** kann eine Schemadatei ausgewählt werden. Zu dieser können dann zu überprüfende JSON-Dateien angegeben werden. PhpStorm kann dann beim Ausfüllen die Angaben im Schema kontrollieren und eventuelle Abweichungen anzeigen.

Für bekannte und öffentliche JSON-Formate, die etwa mit Bibliotheken oder Frameworks verwendet werden, existiert der *JSON Schema Store*¹². Er enthält die Schemas für eine Vielzahl an solchen Formaten. IDEs wie PhpStorm verwenden ihn automatisch und können daher bekannte Formate direkt validieren.

7.3 YAML

YAML – kurz für *YAML Ain't Markup Language* – ist ein menschenlesbares Datenformat und eine vereinfachte Auszeichnungssprache (engl. „lightweight markup language“) [4].

7.3.1 Einführung

YAML hat denselben Einsatzzweck wie XML und JSON: Daten sollen menschenlesbar gespeichert werden. Es wurde als Reaktion auf die Popularität von JSON erdacht, da dieses zwar kompakt und sowohl für Menschen als auch für Maschinen lesbar ist, bei sehr großen Dateien aber aufgrund der vielen geschwungenen Klammern unübersichtlich wird. Der von YAML verfolgte Ansatz ist dabei noch minimalistischer und stellt mehr die Daten in den Vordergrund als das Markup. YAML ist dabei in seiner aktuellsten Version 1.2 ist eine Übermenge von JSON, d. h. jedes JSON-Dokument ist auch ein gültiges YAML 1.2-Dokument (aber nicht umgekehrt).

YAML-Dateien sind ebenfalls Textdokumente, als Endungen kommen `.yaml` oder `.yml` zum Einsatz, wobei ersteres bevorzugt wird.

YAML kommt in vielen Webanwendungen, Frameworks oder Softwarekomponenten zum Einsatz, um etwa Konfigurationen zu hinterlegen oder auch Daten zu übertragen.

7.3.2 Ein YAML-Beispieldokument

Das Knödelrezept aus den Abschnitten 7.1.2 und 7.2.2 soll nun in YAML verfasst werden. Dies kann so aussehen:

```
1 # Tiroler Knödel Rezept
2 %YAML 1.2
3 ---
4 gericht: Der klassische Tiroler Knödel
5 quelle: https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386
6 zutaten:
7   - ingredient: Knödelbrot
```

¹²<https://www.schemastore.org/>

```
8   menge: 350
9   einheit: g
10  - ingredient: Milch
11   menge: 200
12   einheit: ml
13  - ingredient: Eier
14   menge: 4
15   einheit: Stück
16  - ingredient: Speck
17   menge: 100
18   einheit: g
19  - ingredient: Bergsteigerwurst
20   menge: 100
21   einheit: g
22  - ingredient: Butter
23   menge: 20
24   einheit: g
25  - ingredient: Zwiebel
26   menge: 1
27   einheit: Stück
28  - ingredient: Petersilie
29   menge: 10
30   einheit: g
31  - ingredient: Salz
32   menge: 1
33   einheit: Prise
34  - ingredient: Muskatnuss
35   menge: 1
36   einheit: Prise
37  - ingredient: Mehl
38   menge: 80
39   einheit: g
40  - ingredient: Schnittlauch
41   menge: 10
42   einheit: g
43 zubereitung:
44  - >
45    Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.
46  - >
47    Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in
48    zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten
49    und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen
50    Zutaten eine feste, aber nicht zu feuchte Masse kneten.
51  - >
52    Mit befeuchteten Händen daraus Knödel formen.
53  - >
54    Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat der
55    Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die
56    Knödelmasse kneten.
57  - >
58    Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.
59  - >
60    Tiroler Knödel mit Schnittlauch bestreut servieren.
```

Wie ersichtlich ist, verfolgt YAML nicht den klassischen Markup-Ansatz mit Tags. Vielmehr kommt eine vereinfachte Syntax nach dem Schema **Schlüssel: Wert** zum Einsatz. Auffallend ist auch, dass es – umgesetzt auf das XML-Dokument keinen Eintrag

für „Rezept“ gibt und auch kein „Zutat“-Schlüssel vorhanden ist, sondern die Auflistung (die über den Bindestrich – erreicht wird) direkt unter „Zutaten“ steht. Der Grund hierfür ist, dass wie schon erwähnt die Daten im Vordergrund stehen. Eine Verschachtelung unter einem „Rezept“-Schlüssel brächte keinerlei Vorteile, vielmehr sollte das ganze Dokument einfach als Rezept angesehen werden. YAML ist somit viel näher an klassischen Programmier- oder Skriptsprachen, da eigentlich Datenstrukturen (Arrays bzw. assoziative Listen) abgebildet werden.

Ebenso fällt auf, dass es keine Attribute wie bei XML gibt, daher ist die Quelle einfach ein weiterer Eintrag.

7.3.3 YAML-Syntax

YAML verfolgt denselben Ansatz wie JSON, nämlich, dass sich die abzubildenden Daten mit Hilfe von drei Datenstrukturen – Mappings, Sequenzen und Skalaren – darstellen lassen. Wichtig dabei ist auch die Einrückung, die die Struktur vorgibt.

Mappings

Mappings entsprechen den Objekten aus JSON, sind also Name-Wert-Paare. Der Schlüssel wird durch einen Doppelpunkt (:), gefolgt von einem Leerzeichen vom Wert getrennt. Jedes Mapping steht dabei in seiner eigenen Zeile, dies wird in YAML das *Block-Format* genannt.

```
1 name1: wert1
2 name2: wert2
```

Alternativ steht das so genannte *Flow-Format* zur Verfügung, dabei werden die Mappings in geschwungene Klammern durch Beistriche getrennt angegeben:

```
1 { name1: wert1, name2: wert2 }
```

Auch eine mehrzeilige Aufteilung wie in JSON üblich ist möglich:

```
1 {
2   name1: wert1,
3   name2: wert2
4 }
```

Da das Flow-Format aber den Sinn hat, die Daten kompakter als das Block-Format zu repräsentieren, wird in der Regel die einzeilige Variante gewählt (und in weiterer Folge hier auch immer verwendet).

Im Tiroler Knödel-Rezept ist jede Zutat ein Mapping mit drei Schlüsseln. Die Werte sind zwei mal ein String und einmal eine Zahl. Im Block Format wird es wie folgt geschrieben:

```
1 ingredienz: Semmelwürfel
2 menge: 350
3 einheit: g
```

Sequenzen

Sequenzen (engl. „sequences“) sind einfache Arrays. Sie werden durch einen Bindestrich (–) gekennzeichnet, jedes einzelne Element steht bei der Block-Syntax in seiner eigenen Zeile:

```
1 - wert1
2 - wert2
3 - wert3
```

Im Flow-Format werden die Elemente in eckigen Klammern mit Komma getrennt angegeben:

```
1 [ wert1, wert2, wert3 ]
```

Im Tiroler Knödel Rezept ist die Zubereitung ein Array, dessen Werte Zeichenketten sind:

```
1 - Semmelwürfel mit Milch ...
2 - Kleinwürfelig geschnittenen Speck ...
3 - Mit befeuchteten Händen ...
4 - Einen davon als Probeknödel ...
5 - Erst wenn die Masse kompakt genug ist ...
6 - Tiroler Knödel mit Schnittlauch bestreut servieren.
```

Skalare

Skalare sind entweder Text (d. h. Zeichenketten bzw. Strings), Zahlen (Number), boolesche Werte (`true/false`) oder `null`.

Strings: Strings müssen (wie zuvor in den Beispielen schon ersichtlich) *nicht* unter Anführungszeichen gestellt werden, können aber, wenn etwa bereits reservierte Zeichen darin vorkommen. Dabei sind sowohl einfache als auch doppelte Anführungszeichen möglich. Die Werte des Mapping-Beispiels waren etwa Skalare, die ohne Anführungszeichen geschrieben wurden.

Soll längerer Text erfasst werden, können zwei Stile verwendet werden: literal oder folded. Ersterer wird mit einem Pipe-Zeichen (`|`) begonnen und Zeilenumbrüche werden so beibehalten, wie sie im Text aufscheinen. Der zweite Stil beginnt mit einer spitzen Klammer (`>`), Zeilenumbrüche werden einfach auf Leerzeichen umgewandelt. Bei beiden Stilen wird das jeweilige Zeichen in die erste Zeile gesetzt und der Text dann um zwei Leerzeichen eingerückt in die darauffolgenden:

```
1 - |
2   Dieser Text bricht
3   genauso um, wie er
4   hier geschrieben ist.
5 - >
6   Dieser Text ist im
7   Prinzip ein langer
8   String und hier nur
9   zwecks Lesbarkeit auf
10  mehrere Zeilen aufgeteilt.
```

Da im Tiroler Knödel-Rezept die Zubereitungsschritte lange sind, kommt hier der folded Stil zum Einsatz:

```
1 - >
2   Semmelwürfel mit Milch ...
3 - >
4   Kleinwürfelig geschnittenen Speck ...
5 - >
```

```

6 Mit befeuchteten Händen ...
7 - >
8 Einen davon als Probeknödel ...
9 - >
10 Erst wenn die Masse kompakt genug ist...
11 - >
12 Tiroler Knödel mit Schnittlauch bestreut servieren.

```

Zahlen: Zahlen werden ebenfalls ohne Anführungszeichen geschrieben. YAML erlaubt hier (im Vergleich zu JSON) nicht nur Dezimalzahlen, sondern auch Oktalzahlen (mit führender Null, z.B. 0755) und Hexadezimalzahlen (mit führendem 0x, z.B. 0xFF). Ebenso können Zahlen mit dem Underscore (_) als Tausendertrennzeichen verwendet werden (z.B. 1_000_000), um sie optisch besser darzustellen.

```
1 menge: 350
```

true, false und null: Die Wahrheitswerte und null werden analog zu JSON verwendet.

```

1 aus_österreich: true
2 vegetarisch: false
3 bewertung: null

```

Struktur und Direktiven

YAML-Dokumente werden wie in JSON in eine eigene Datei geschrieben. Dabei ist es jedoch möglich, dass eine YAML-Datei mehrere Dokumente enthält. Diese werden mit drei Bindestrichen (---) von einander getrennt. Dieses Zeichen ist also der Beginn eines neuen Dokuments mit Inhalt. Ist nur ein Dokument in der Datei, können die drei Bindestriche entfallen.

Alternativ können vor dem eigentlichen Dokument auch so genannte Direktiven stehen. Diese sind kein Inhalt, sondern ähnlich wie XML-Processing Instructions Anweisungen an etwaige Parser. Direktiven in YAML werden mit dem Prozentzeichen (%) geschrieben. Die Direktiven werden dann mit --- vom eigentlichen Inhalt getrennt.

Um das Ende eines Dokuments zu markieren, werden drei Punkte (...) verwendet. Diese sind jedoch nicht zwingend notwendig und können auch entfallen.

Das folgende Beispiel besteht aus zwei Abschnitten, zu Beginn steht eine Direktive zur YAML-Versionsnummer:

```

1 %YAML 1.2
2 ---
3 rezept: Tiroler Knödel
4 schwierigkeit: einfach
5 vegetarisch: false
6 ...
7 ---
8 rezept: Käsespätzle
9 schwierigkeit: einfach
10 vegetarisch: true
11 ...

```

Kommentare

YAML unterstützt im Gegensatz zu JSON Kommentare. Diese werden mit der Raute (#) verfasst und sind einzeilig.

```
1 # Tiroler Knödel Rezept
2 %YAML 1.2
3 ---
4 gericht: Der klassische Tiroler Knödel
```

Einrückungen

Einrückungen spielen bei den Block-Stilen in YAML eine große Rolle. Dabei dürfen ausschließlich Leerzeichen und keine Tabulatoren verwendet werden. Die Anzahl ist dabei freigestellt, in der Regel kommen zwei oder vier Leerzeichen für eine Einrückungsstufe zum Einsatz. Eine Einrückung zeigt dabei eine Hierarchieebene in der Datenstruktur an, diese wird wieder beendet, sobald ein Element folgt, das weniger eingerückt ist. Im folgenden wird ein Mapping mit einer Sequenz als Wert, die wiederum einzelne Mappings enthält gezeigt. Jede Stufe ist mit 2 Leerzeichen eingerückt.

```
1 zutaten:
2   - ingredient: Semmelwürfel
3     menge: 350
4     einheit: g
5   - ingredient: Milch
6     menge: 200
7     einheit: ml
```

Weitere Merkmale

YAML beherrscht noch weitere Fähigkeiten wie etwa die explizite Angabe von Datentypen oder auch das Setzen von Aliases, einem Referenz-Mechanismus, falls sich Werte wiederholen. Für diese Dinge sei auf die YAML-Dokumentation unter [4] verwiesen.

7.3.4 YAML validieren

Auch YAML bietet die Möglichkeit einer Validierung. Dabei kommt auch JSON Schema zum Einsatz, da YAML dieselben Datenstrukturen und -typen verwendet. Wiederum gilt auch hier, dass das Schema nicht im Dokument angegeben wird, sondern über Tools eingebunden wird. Der Prozess ist dabei identisch wie für JSON in Abschnitt 7.2.6.

Das Schema für das Tiroler Knödel-Rezept in YAML ist also genau dasselbe wie das in Abschnitt 7.2.5 gezeigte.

7.4 Weiterführende Literatur

Das Angebot an Fachliteratur zum Thema XML ist überdurchschnittlich groß. Demnach ist es auch nicht einfach, *das* Standardwerk schlechthin zu empfehlen. Ein Buch, das schon länger am Markt ist, ist [3]. Es bietet einen soliden Überblick über die Grundlagen und verschiedenen Anwendungsgebiete von XML und ist mittlerweile schon in seiner

dritten Auflage vorhanden. Diese ist jedoch schon aus dem Jahr 2004, wobei es bei XML selbst in den letzten Jahren keine Neuerungen mehr gab.

Ebenfalls vom selben Autor ist [2], welches jedoch mehr mit praxisbezogenen Tipps aufwartet.

Wer ein deutsches Buch zum Thema XML sucht, dem sei [5] ans Herz gelegt. Das Buch hat 2018 seine neunte Auflage bekommen, ist also im Vergleich zu anderen Werken aktuell und enthält alle wichtigen Themen auch für Einsteiger gut aufbereitet.

Literatur zu JSON und YAML hingegen ist spärlich. Um die Formate zu erlernen, bieten sich die offiziellen Dokumentationen [1, 4] an, des weiteren finden sich online diverse Tutorials, die beide Formate erklären.

Quellenverzeichnis

- [1] European Computer Manufacturers Association. *The JSON Data Interchange Syntax*. ECMA-404. Version 2.0. Dez. 2017. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-404/> (siehe S. 7-17, 7-31).
- [2] Elliotte Rusty Harold. *Effective XML. 50 Specific Ways to Improve Your XML*. Amsterdam, Niederlande: Addison-Wesley Longman, 1. Sep. 2003 (siehe S. 7-7, 7-31).
- [3] Elliotte Rusty Harold und W. Scott Means. *XML in a Nutshell. A Desktop Quick Reference*. 3. Aufl. Sebastopol, USA: O'Reilly, 2. Nov. 2004 (siehe S. 7-2, 7-5, 7-30).
- [4] Ingy döt Net u. a. *YAML Ain't Markup Language (YAML™) version 1.2*. Revision 1.2.2. 1. Okt. 2021. URL: <https://yaml.org/spec/1.2.2/> (siehe S. 7-25, 7-30, 7-31).
- [5] Helmut Vonhoegen. *XML. Einstieg, Praxis, Referenz*. 10. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Dez. 2023 (siehe S. 7-31).
- [6] World Wide Web Consortium. *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. W3C Recommendation. 26. Nov. 2008. URL: <https://www.w3.org/TR/xml/> (siehe S. 7-1, 7-7).
- [7] World Wide Web Consortium. *Extensible Markup Language (XML) 1.1 (Second Edition)*. W3C Recommendation. Edited in place 29 September 2006. 16. Aug. 2006. URL: <https://www.w3.org/TR/xml11/> (siehe S. 7-7).
- [8] World Wide Web Consortium. *XML Schema Part 1: Structures Second Edition*. W3C Recommendation. 28. Okt. 2004. URL: <https://www.w3.org/TR/xmlschema-1/> (siehe S. 7-13).